Advanced AI Medical Intelligence Platform

Project Report — Chest X-Ray Pneumonia Detection with Explainable AI and LLM-Assisted Reporting

Prepared for: SN Matrix Software Pvt.Ltd Git-hub: https://github.com/Vig1259/Advanced-AI-Medical-Intelligence-Platform

Disclaimer: This is a research/demo system built for a technical assessment. It is not a certified medical device and must not be used for real clinical diagnosis. All AI outputs require review by a licensed physician.

1. Project Objective

The objective of this project is to develop an end-to-end AI-powered medical imaging platform for the automated detection of pneumonia from chest X-ray images. The system leverages a deep learning model to classify X-ray images as NORMAL or PNEUMONIA, while incorporating Explainable Artificial Intelligence (XAI) using Grad-CAM to provide visual explanations of the model's predictions. Additionally, the platform integrates a Large Language Model (Google Gemini) to generate structured, AI-assisted preliminary radiology reports based on the model's output.

2. System Architecture

The system follows a layered architecture: a Streamlit frontend for image upload and results visualization; a FastAPI REST backend exposing prediction and history endpoints; a PyTorch inference service (DenseNet121 backbone + Grad-CAM); a Gemini-based report generation module; and a SQLAlchemy-backed database (SQLite by default, Postgres-ready) for persisting every prediction. Components communicate over HTTP/JSON, and the whole stack is containerized via Docker Compose.

Request flow:

- The user registers or logs into the application through the Streamlit interface.
- The user uploads a chest X-ray image for analysis.
- The image is sent to the FastAPI backend through a secure REST API.
- The DenseNet121 model preprocesses the image and predicts whether it is NORMAL or PNEUMONIA, along with confidence scores.
- Grad-CAM generates a heatmap highlighting the image regions that most influenced the prediction.
- The prediction results and Grad-CAM findings are forwarded to the Google Gemini API to generate a structured preliminary radiology report.
- The prediction, confidence scores, Grad-CAM visualization, AI-generated report, and timestamp are stored in the database.
- The FastAPI backend returns to complete response to the Streamlit frontend, when the user can view the prediction explanation, generated report, and access previous prediction history.

3. Dataset and Model

Dataset: Kermany et al. "Chest X-Ray Images (Pneumonia)" dataset (Guangzhou Women and Children's Medical Center), publicly available via Kaggle. It contains labeled pediatric chest X-rays in two classes: NORMAL and PNEUMONIA, with a known class imbalance (~3x more PNEUMONIA than NORMAL cases).

Model: DenseNet121 pretrained on ImageNet, fine-tuned for binary classification. DenseNet's dense connectivity pattern improves gradient flow and feature reuse, and it underlies CheXNet, a well-known chest radiograph classification benchmark. Training uses class-weighted cross-entropy loss to address imbalance, a short backbone-frozen warmup phase for the classifier head, data augmentation (flips, small rotations, color jitter), and a ReduceLROnPlateau learning-rate schedule with best-checkpoint selection by validation AUC.

Test Set Results: The model was trained for 15 epochs (2 epochs of classifier-head-only warmup followed by full fine-tuning) and evaluated on the held-out test set (234 NORMAL, 390 PNEUMONIA images, n=624). Final test metrics: loss = 0.3127, accuracy = 88.94%, ROC-AUC = 0.9679.

Class	Precision	Recall	F1-score	Support
NORMAL	0.98	0.72	0.83	234
PNEUMONIA	0.85	0.99	0.92	390
Accuracy			0.89	624
Macro avg	0.92	0.86	0.87	624
Weighted avg	0.90	0.89	0.88	624

Confusion Matrix (test set, n=624):

	Predicted NORMAL	Predicted PNEUMONIA
Actual NORMAL	168	66
Actual PNEUMONIA	3	387

Interpretation: The model achieves strong overall discrimination (AUC = 0.968) and, critically for a screening tool, very high PNEUMONIA recall (0.99) -- only 3 of 390 actual pneumonia cases were missed (false negatives). This comes at the cost of lower NORMAL recall (0.72): 66 of 234 healthy X-rays were flagged as PNEUMONIA (false positives). This precision/recall tradeoff is a direct, intended consequence of the class-weighted loss function used during training (Section 8), which was deliberately biased toward minimizing missed pneumonia cases over minimizing false alarms -- a reasonable choice for a
screening-support tool, where a missed diagnosis is more costly than a false alarm that gets corrected on physician review. This tradeoff, and the fact that a false positive rate this size would need to be reduced before any real clinical use, is exactly why every prediction is paired with a Grad-CAM visualization and routed through physician review rather than used as a standalone diagnosis.

4. Explainable AI (Grad-CAM)

Grad-CAM (Selvaraju et al., 2017) was implemented from scratch (not via a third-party XAI library) in app/ml/gradcam.py. Forward and backward hooks are registered on the last normalization layer of the DenseNet121 feature extractor. Gradients of the predicted class score with respect to the feature maps are global-average-pooled into per-channel importance weights, which are used to compute a weighted combination of the feature maps, followed by ReLU and min-max normalization. The resulting heatmap is resized and alpha-blended over the original X-ray to visually indicate which lung regions most influenced the model's decision.

5. LLM-Assisted Report Generation

The Google Gemini API (default model: gemini-2.5-flash, configurable) is used to convert the structured prediction output into a readable draft report. The system prompt constrains the model to a fixed structure (AI Impression, Key Observations, Confidence & Limitations, Suggested Next Steps), instructs it to hedge appropriately rather than assert a definitive diagnosis, and forbids inventing findings beyond the provided data. Every report is appended with an explicit non-diagnostic disclaimer. If no API key is configured, or the API call fails, the system falls back to deterministic template.

6. Security: Authentication & Rate Limiting

Authentication: JWT-based auth (app/auth.py, app/routers/auth.py) protects all prediction and history endpoints. Passwords are hashed with bcrypt (via passlib) before storage plaintext passwords are never persisted. On login, a signed JWT (1-hour default expiry) is issued and must be sent as a Bearer token on subsequent requests. Every prediction record is tagged with its owning user, and all history queries are scoped to request. User so one user can never read or delete another user's data, even by guessing a record ID (a mismatched ID returns 404, not 403, to avoid confirming which IDs exist).

Rate limiting: Implemented with slowapi, keyed by client IP. /predictis limited to 10 requests/minute to protect LLM API quota and inference compute from abuse; /auth/loginis limited to 5 requests/minute to slow brute-force password guessing. Requests over the limit receive an immediate 429 response before reaching any route logic.

Known gaps: rate limiting is IP-based rather than per-account, so it can be shared unfairly across users behind the same NAT or bypassed by an attacker rotating IPs. There is no password reset flow, email verification, or role-based access control all authenticated users currently have identical permissions. The JWT signing secret auto-generates per-process if not explicitly set, which is fine for a single dev instance but would invalidate all sessions on restart and is unsuitable for multi-instance production without a fixed, shared JWT_SECRET_KEY.

7. REST API Design

Endpoint	Method	Auth	Description
/auth/register	POST	No	Create a new user account (bcrypt-hashed password)
/auth/login	POST	No	Authenticate; returns a JWT access token (5/min rate limit)
/predict	POST	Yes	Upload an X-ray; returns prediction, confidence, Grad-CAM overlay, LL
/history	GET	Yes	List the caller's own past predictions (paginated, filterable by class)
/history/{id}	GET	Yes	Full detail of a single prediction record owned by the caller
/history/{id}	DELETE	Yes	Delete a prediction record owned by the caller

Built with FastAPI, with Pydantic schemas for request/response validation and automatic OpenAPI/Swagger documentation at /docs.

8. Database Design

A single prediction_history table (SQLAlchemy ORM, app/models_db.py) stores: a UUID primary key, original filename, predicted class, confidence, the full class-probability distribution (JSON), the path to the saved Grad-CAM overlay image, the generated LLM report text, and a timestamp. SQLite is used by default for zero-config local/demo use the connection string is externalized via DATABASE_URLso swapping to PostgreSQL for production requires no code changes.

9. Setup, Training, and Reproduction Steps

- Clone the repository and install dependencies: pip install -r requirements.txt
- Download the Kaggle "Chest X-Ray Images (Pneumonia)" dataset into ./data/ following the train/val/test/NORMAL/PNEUMONIA layout
- Train the model: python training/train.py --data-root data --epochs 15
- Evaluate on the test set: python training/evaluate.py checkpoint models/chest_xray_densenet121.pt
- Set GEMINI_API_KEYand a fixed JWT_SECRET_KEYin .env
- Run the API: uvicorn app.main:app --reload
- Register a user and log in via /docs(Swagger UI's Authorize button) or the frontend
- Run the frontend: streamlit run frontend/streamlit_app.py
- Or run everything via Docker: docker compose up –build.

10. Evaluation Criteria Coverage

Criterion	Where addressed
Deep Learning model performance	training/train.py, training/evaluate.py (Sec. 3, 9)
Code quality / project structure	Modular app/, training/, frontend/, tests/ layout (Sec. 2)
Explainable AI (Grad-CAM)	app/ml/gradcam.py (Sec. 4)
LLM integration	app/llm/report_generator.py (Sec. 5)
Security (auth, rate limiting)	app/auth.py, app/routers/auth.py, app/rate_limit.py (Sec. 6)
API development	app/main.py, app/routers/ (Sec. 7)
Database design	app/models_db.py, app/database.py (Sec. 8)
Web application	frontend/streamlit_app.py
Documentation	README.md, this report
Deployment	Dockerfile, docker-compose.yml
System design / best practices	Auth, rate limiting, config via env vars, tests/, error handling, disclaimers

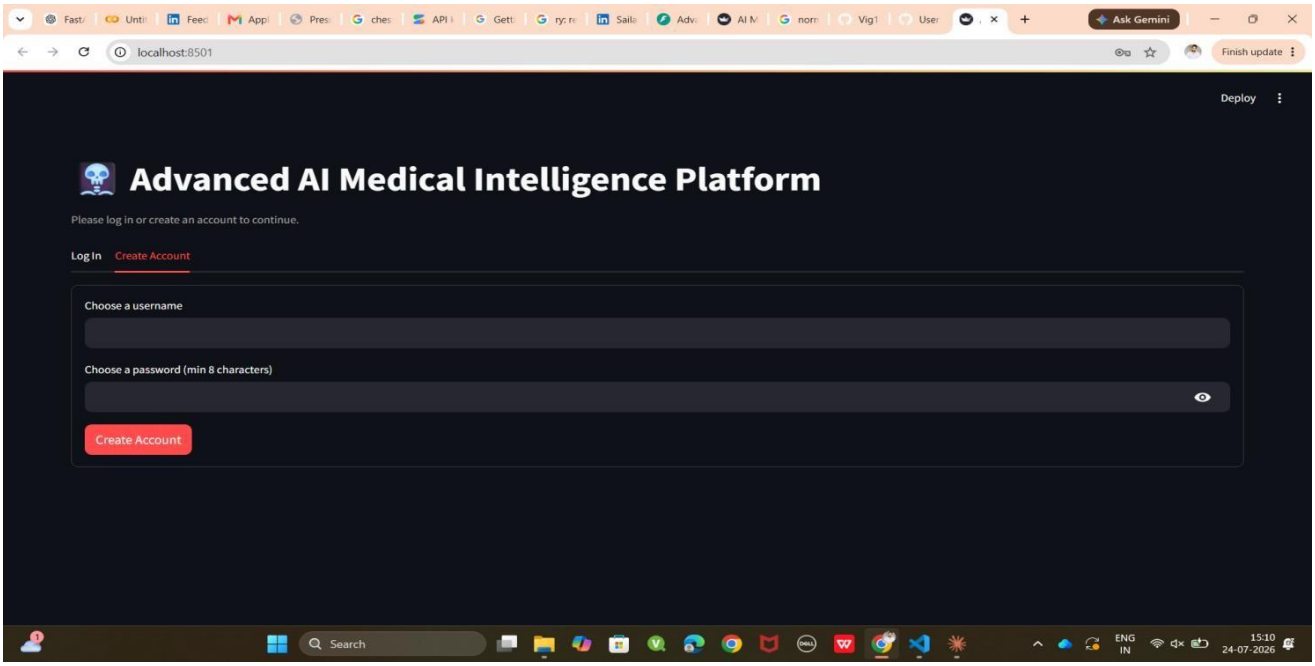
11. Known Limitations and Future Work

- NORMAL-class recall is only 0.72 (66 of 234 healthy X-rays misclassified as PNEUMONIA) -- a direct result of prioritizing pneumonia sensitivity via class-weighted loss (Sec. 3). This false-positive rate would need to be reduced before any real clinical deployment.
- The Kaggle dataset consists entirely of pediatric patients (ages 1-5) from a single medical center; performance on adult chest X-rays or images from other imaging equipment/populations is unverified and may differ substantially.
- Grad-CAM region description fed to the LLM is currently a simple heuristic rather than computed heatmap centroid/quadrant analysis.
- Rate limiting is IP-based, not per-user-account (Sec. 6) -- shareable across users on the same network, bypassable by IP rotation.
- No password reset, email verification, or role-based access control; all authenticated users have identical permissions (Sec. 6).
- SQLite used for demo purposes; recommend PostgreSQL for concurrent production workloads.
- Only JPEG/PNG supported; no DICOM parsing.

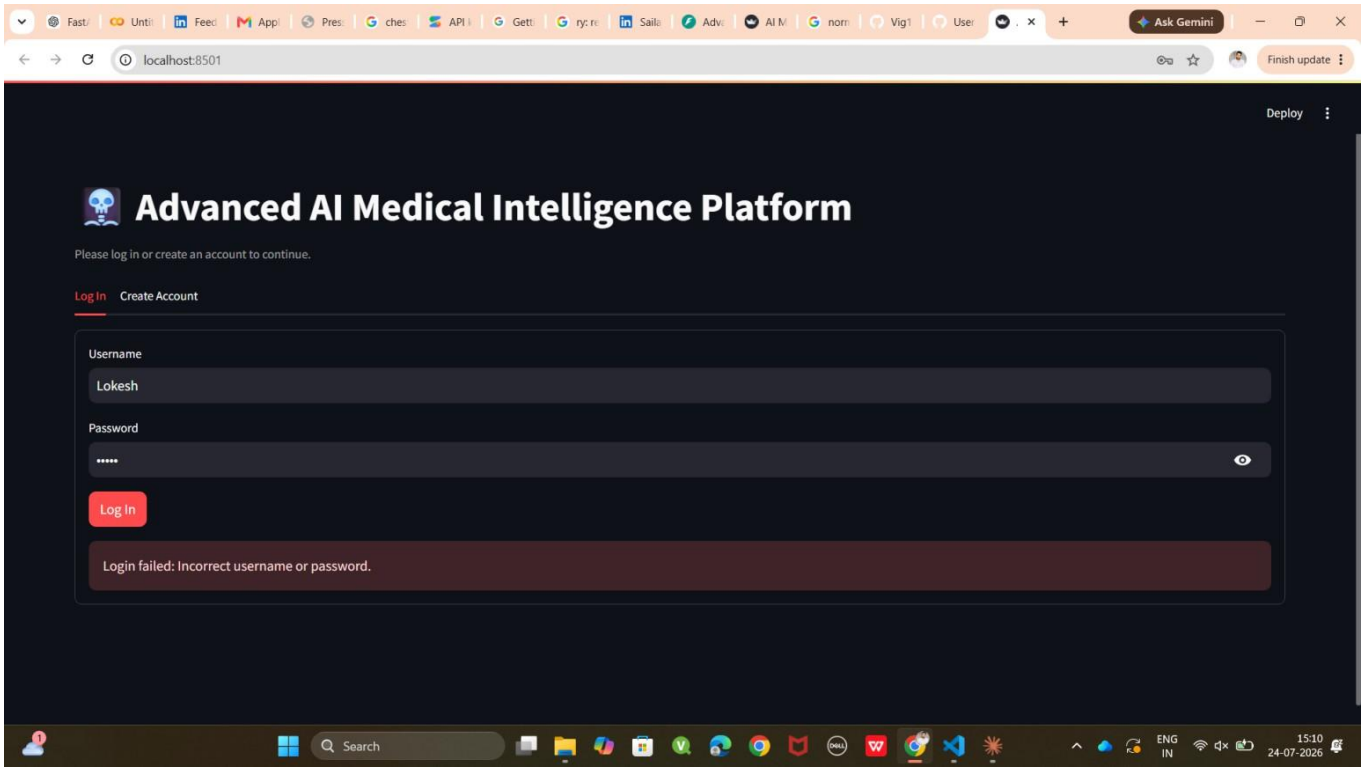
12. Results and Snapshots

```
Test results: loss=0.3127 acc=0.8894 auc=0.9679

Classification report:
              precision    recall  f1-score   support

      NORMAL       0.98      0.72      0.83       234
   PNEUMONIA       0.85      0.99      0.92       390

    accuracy                           0.89       624
   macro avg       0.92      0.86      0.87       624
weighted avg       0.90      0.89      0.88       624

Confusion matrix:
 [[168  66]
 [  3 387]]
```

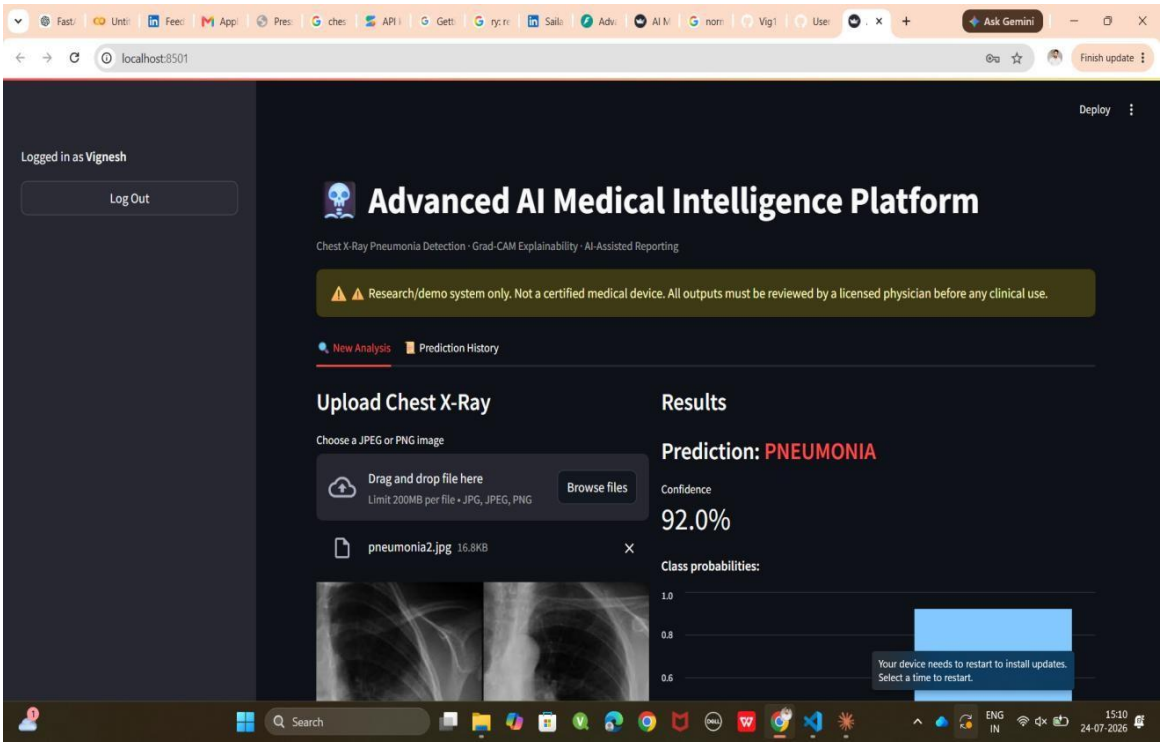
Test Results



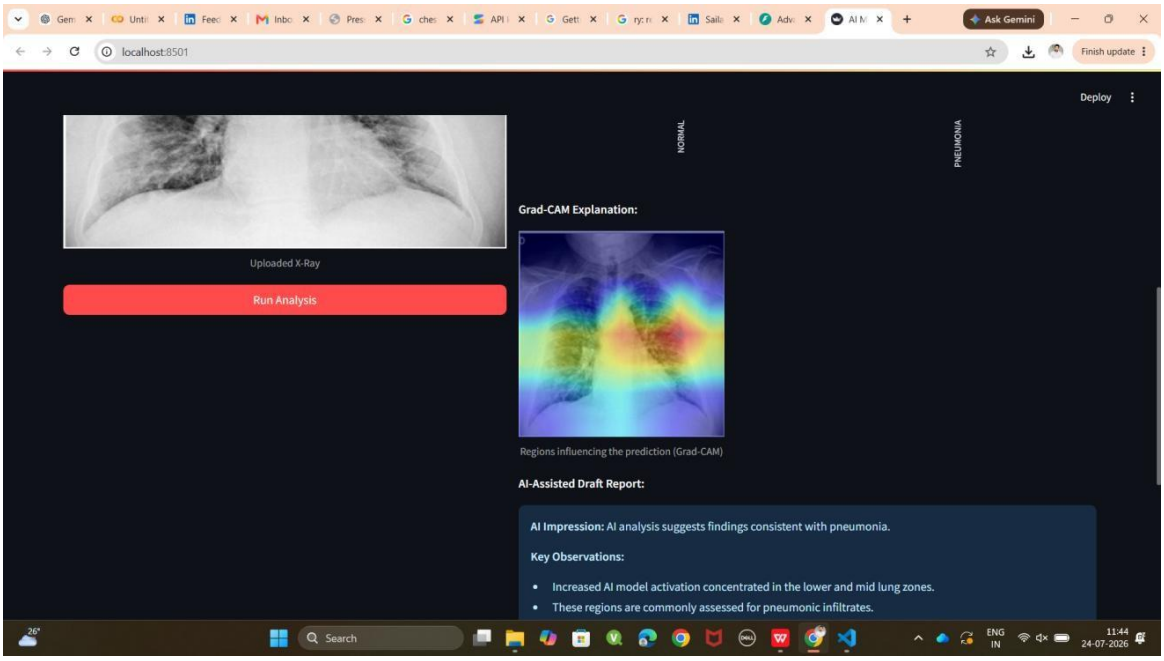
User Authentication Page



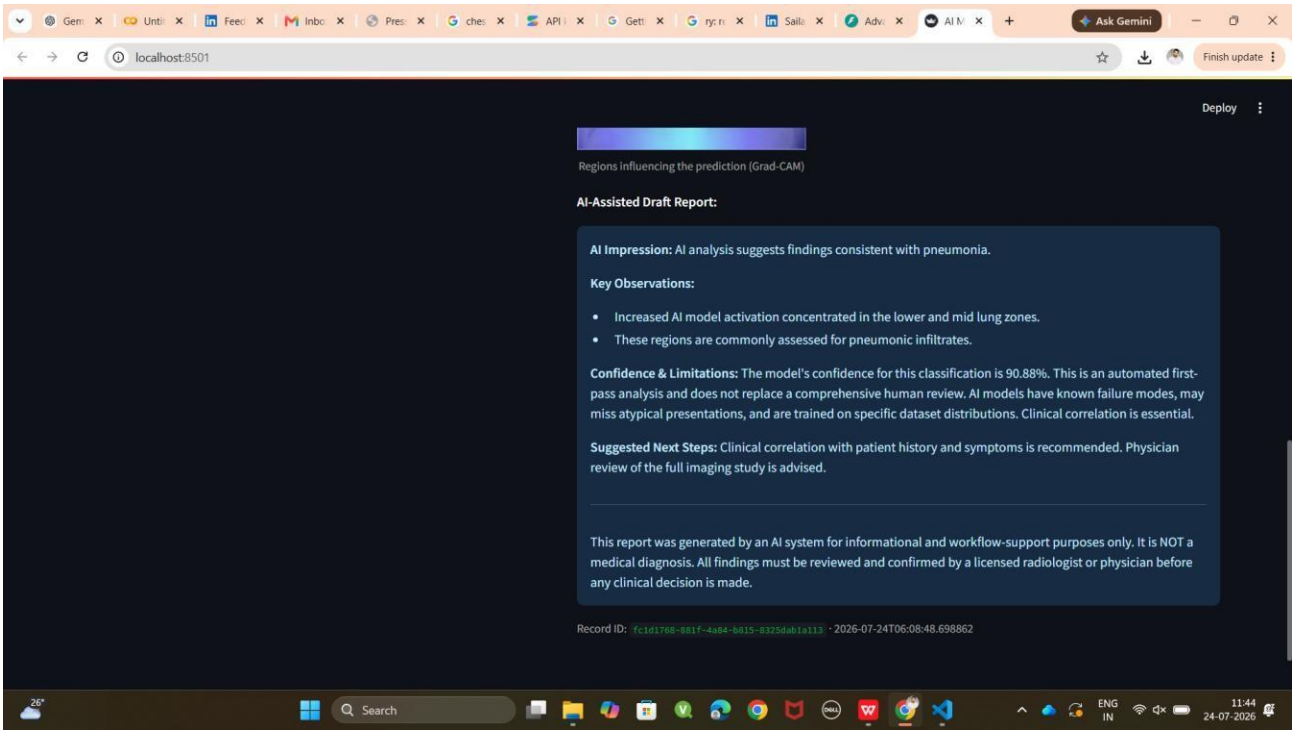
Invalid Credentials



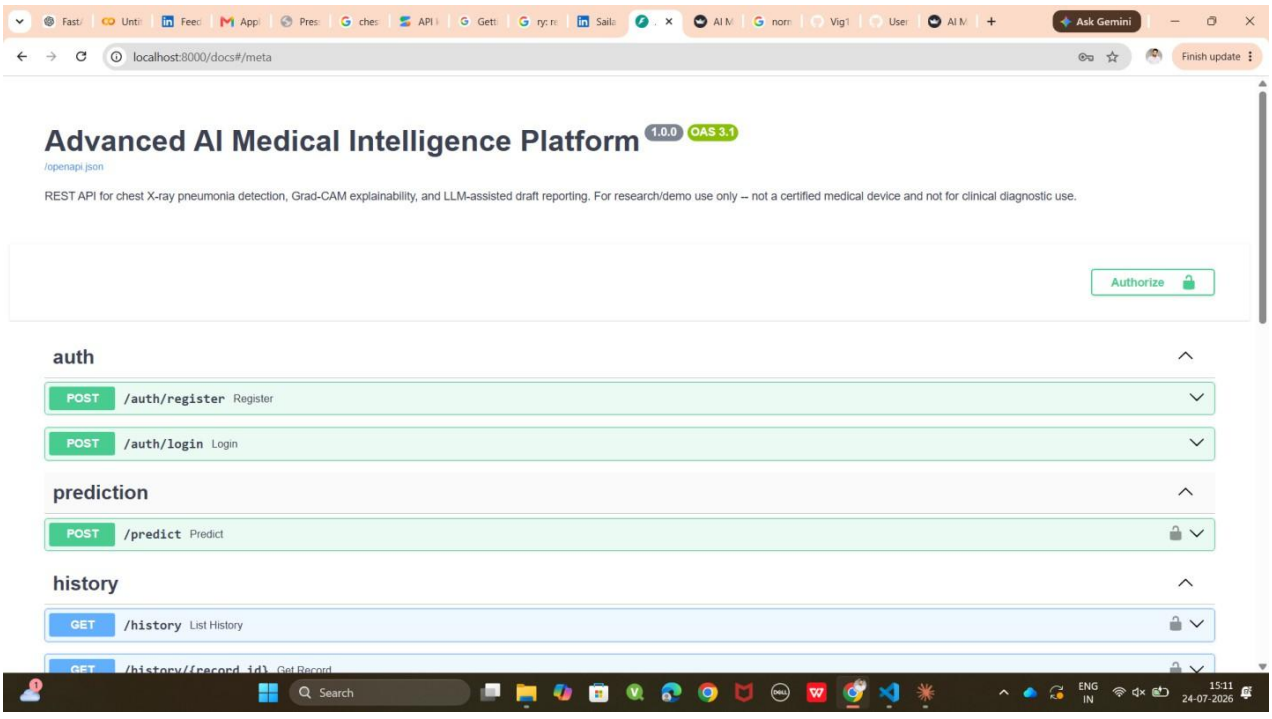
Analysis Page



Grad-Cam Analysis



LLM Report Generation



Backend

13. References

- Kermany, D. et al. "Chest X-Ray Images (Pneumonia)" dataset, Guangzhou Women and Children's Medical Center (via Kaggle).
- Huang, G. et al. "Densely Connected Convolutional Networks" (DenseNet), CVPR 2017.
- Selvaraju, R. R. et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization", ICCV 2017.
- Rajpurkar, P. et al. "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning", 2017.